



## Distributed System Monitor: Tracking Events and Capturing Global State

Course: Distributed Computing (CCZG 526)

Assignment: Lab Assignment I

Group Number: Group-2

Video Recording Link: <https://drive.google.com/file/d/1DN8kpzoTmc0XPLNZMKuC5BFI6iqj6C03/view?usp=sharing>

GitHub Repo - <https://github.com/kashifxyz/bits-food-delivery-distributed-computing-assignment>

(All team members connected virtually and individually explained their own contribution; the call is recorded and uploaded to Google Drive.)

### Contribution Table

| Sl. No. | Name (as appears in Canvas) | ID No.      | Contribution                                                                                                                                                                                                                                                                                                                                                                                                 |
|---------|-----------------------------|-------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1       | Anirudh Gudipati            | 2025MT03510 | Built vector_clock.py, the thread-safe vector clock used by all 5 processes, with a 9-test unit suite. Fixed a Windows UTF-8 crash, two snapshot bugs (marker ordering and a snapshot-ID mismatch stalling Snapshot 2), compiled this submission                                                                                                                                                             |
| 2       | Shaik Baleeghuddin Kashif   | 2026NS03020 | Built p0.py, the coordinator, handling order dispatch, snapshot initiation, global state collection, causal consistency checks, concurrency analysis, an interactive CLI, and the automated demo flow. Also fixed multi-node networking and wrote the project docs.                                                                                                                                          |
| 3       | Vinjanampati Sri Harsha     | 2026NS03002 | Implemented p1.py (Pizza Palace) and p2.py (Burger Hub), covering order confirmation, food preparation internal events, delivery dispatch to P3/P4, and MARKER forwarding through the shared snapshot module.                                                                                                                                                                                                |
| 4       | Nerella Chandra Hasitha     | 2025MT03502 | Implemented P3/P4 delivery partners, handling the full lifecycle (picked up, in transit, delivered), sending delivery confirmations to P0, and handling MARKER/STATE messages through the shared snapshot module.                                                                                                                                                                                            |
| 5       | Abhirup Bhattacharjee       | 2025MT03584 | Implemented snapshot.py, encapsulating the Chandy-Lamport algorithm in a reusable class that handles snapshot initiation, marker propagation, local state recording, channel state tracking, and state collection. Integrates with the vector clock module and processes MARKER, STATE, and application messages, enabling consistent distributed state capture over sockets.                                |
| 6       | Sohail Quazi                | 2025MT03580 | Handled infrastructure and deployment: wired up node IPs into hosts.cfg, built preflight and connectivity checks, diagnosed via nmap that the lab firewall blocks ports 5001–5005 (only 22/3389 open), so the system ran as five processes on one node, and built the demo launcher plus a script generating the consistency proof and logs for submission. Also handled the demo intro and video recording. |

### Language Used for the Implementation

Python 3 (standard library sockets for inter-process communication; colorama for coloured terminal audit logs).



## 1. System Overview

The system models a distributed online food delivery platform as 5 independent processes communicating over TCP sockets, with no shared/global clock:

P0 (Coordinator) dispatches ORDER to P1 (Pizza Palace) and ORDER to P2 (Burger Hub).

P1 sends DELIVERY to P3 (Fleet Runner A). P2 sends DELIVERY to P4 (Fleet Runner B).

- P0 dispatches Order #101 to P1 and Order #102 to P2 (hardcoded, one-directional — P1 and P2 never communicate with each other).
- P1 always forwards its delivery to P3; P2 always forwards its delivery to P4.
- Each process maintains its own Vector Clock (`vector_clock.py`) of size 5, updated on every internal, send, and receive event.
- P0 triggers two Chandy-Lamport global snapshots during the run, propagated via MARKER messages and collected via STATE messages back to P0.

## Project File Structure

Each process runs as its own file, but the two cross-cutting concerns — vector clocks and the snapshot algorithm — live in one shared module `apiece`, imported by every process rather than reimplemented per file:

| File                              | Role                                                                                                                                                                                                                                                                                                                                                                                    |
|-----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>p0.py</code>                | Central Order Processor / Coordinator. Dispatches Order #101 to P1 and Order #102 to P2, initiates both Chandy-Lamport snapshots (MARKER broadcast), collects STATE from all 5 processes, runs the causal consistency check ( $V_i[j] \leq V_j[j]$ for every pair), and provides both an interactive CLI and an automated end-to-end demo mode ( <code>--auto</code> ).                 |
| <code>p1.py</code>                | Pizza Palace (Restaurant A). Receives ORDER #101 from P0, runs order-confirmation and food-preparation as internal events, sends DELIVERY to P3, and forwards/handles snapshot MARKER messages on $P0 \rightarrow P1$ and $P1 \rightarrow P3$ channels.                                                                                                                                 |
| <code>p2.py</code>                | Burger Hub (Restaurant B). Receives ORDER #102 from P0, runs order-confirmation and food-preparation as internal events, sends DELIVERY to P4, and forwards/handles snapshot MARKER messages on $P0 \rightarrow P2$ and $P2 \rightarrow P4$ channels.                                                                                                                                   |
| <code>p3.py</code>                | Fleet Runner A. Receives the DELIVERY handoff from P1, runs pickup / in-transit / delivered as internal events, and sends DELIVERY_COMPLETE back to P0. Participates in snapshot recording on the $P1 \rightarrow P3$ channel.                                                                                                                                                          |
| <code>p4.py</code>                | Fleet Runner B. Receives the DELIVERY handoff from P2, runs pickup / in-transit / delivered as internal events, and sends DELIVERY_COMPLETE back to P0. Participates in snapshot recording on the $P2 \rightarrow P4$ channel.                                                                                                                                                          |
| <code>vector_clock.py</code>      | Shared, thread-safe Vector Clock library (size-5 vector) imported by all 5 processes — not reimplemented per process. Exposes <code>increment()</code> for internal events, <code>send_event()</code> to stamp outgoing messages, <code>receive_event()</code> to merge an incoming clock (component-wise max, then increment), and <code>is_concurrent()</code> to compare two clocks. |
| <code>test_vector_clock.py</code> | Unit test suite for <code>vector_clock.py</code> — the 9 tests behind TC01–TC09 in the Test Cases table (initial state, local increment, send/receive/internal event correctness, concurrent vs. causal detection, and thread safety under concurrent access).                                                                                                                          |
| <code>snapshot.py</code>          | Shared ChandyLamportSnapshot class imported by all 5 processes. Implements marker propagation, local state recording on first marker receipt, in-transit channel-message recording ( <code>record_channel_message</code> ) for messages that arrive mid-snapshot, and reporting collected state back to P0.                                                                             |
| <code>config/hosts.cfg</code>     | Network configuration file mapping each process (P0–P4) to its port and node IP. Resolved at startup so the same code runs unmodified whether all 5 processes run on a single machine or are spread across multiple Prayogshala cluster nodes.                                                                                                                                          |

## 2. Vector Clock Rules

Each process  $P_i$  maintains a vector  $V_i = [v_0, v_1, v_2, v_3, v_4]$ , initialized to all zeros.

- Internal event:  $V_i[i] += 1$
- Send event:  $V_i[i] += 1$ , attach  $V_i$  to the outgoing message
- Receive event (message carries  $V_{msg}$ ): for all  $k$ ,  $V_i[k] = \max(V_i[k], V_{msg}[k])$ , then  $V_i[i] += 1$

Two events/processes are concurrent ( $A \parallel B$ ) if neither  $V_A \leq V_B$  nor  $V_B \leq V_A$  holds.

## 3. Chandy-Lamport Snapshot Algorithm

Implemented in `snapshot.py`'s `ChandyLamportSnapshot` class, shared by all 5 processes:

1. Initiator ( $P_0$ ): sends MARKER along all outgoing channels (to  $P_1, P_2$ ), starts recording on incoming channels.
2. First marker receipt at  $P_j$ : records local state, marks the channel the marker arrived on as empty, forwards MARKER along its own outgoing channels ( $P_1 \rightarrow P_3, P_2 \rightarrow P_4$ ), starts recording any other incoming channels, sends STATE back to  $P_0$ .
3. Subsequent marker on an already-recording channel: stops recording that channel; its captured contents become the channel's final recorded state.
4.  $P_0$  collects STATE from all 5 processes and prints the global snapshot report.

## 4. Test Cases

| ID   | Test Name                         | Purpose                                                                                                                                                                             | Result                                                                                                                                        |
|------|-----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| TC01 | Initial Clock State               | Vector clock initializes to $[0, 0, 0, 0, 0]$                                                                                                                                       | PASS                                                                                                                                          |
| TC02 | Local Increment                   | <code>increment()</code> advances only the owning process's slot                                                                                                                    | PASS                                                                                                                                          |
| TC03 | Send Event                        | <code>send_event()</code> increments and returns the clock attached to outgoing messages                                                                                            | PASS                                                                                                                                          |
| TC04 | Receive Event                     | <code>receive_event()</code> merges via component-wise max, then increments                                                                                                         | PASS                                                                                                                                          |
| TC05 | Internal Event                    | <code>internal_event()</code> advances the local slot independently                                                                                                                 | PASS                                                                                                                                          |
| TC06 | Concurrent Event Detection        | Two causally-unrelated events are correctly identified as concurrent                                                                                                                | PASS                                                                                                                                          |
| TC07 | Non-Concurrent (Causal) Detection | A happens-before pair is correctly identified as not concurrent                                                                                                                     | PASS                                                                                                                                          |
| TC08 | Full Order Causal Flow            | End-to-end vector clock propagation $P_0 \rightarrow P_1 \rightarrow P_3$                                                                                                           | PASS                                                                                                                                          |
| TC09 | Thread Safety                     | 500 concurrent increments across 5 threads all counted correctly                                                                                                                    | PASS                                                                                                                                          |
| TC10 | Order 1 End-to-End Delivery       | $P_0$ dispatches Order #101 $\rightarrow P_1$ confirms/prepares $\rightarrow$ DELIVERY to $P_3 \rightarrow P_3$ picks up/transits/delivers $\rightarrow$ DELIVERY_COMPLETE to $P_0$ | PASS — $P_0$ dispatched at Clock $[2, 0, 0, 0, 0]$ ; "[DELIVERY COMPLETE] Order fulfilled via $P_3$ " confirmed at Clock $[11, 8, 6, 7, 2]$ . |

| ID   | Test Name                             | Purpose                                                                                                                          | Result                                                                                                                                                                                                                          |
|------|---------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| TC11 | Order 2 End-to-End Delivery           | Same flow for Order #102 via P0→P2→P4                                                                                            | PASS — P0 dispatched at Clock [4, 0, 0, 0, 0]; "[DELIVERY COMPLETE] Order fulfilled via P4" confirmed at Clock [13, 8, 8, 7, 7].                                                                                                |
| TC12 | Concurrent Internal Events (P1    P2) | P1 and P2 independently process their orders with no channel between them; their internal-event clocks are mutually incomparable | PASS — at SNAPSHOT_1, $V_{P1} = [5, 7, 0, 0, 0]$ and $V_{P2} = [5, 0, 5, 0, 0]$ . Neither dominates: $V_{P1}[2] = 0 < V_{P2}[2] = 5$ but $V_{P2}[1] = 0 < V_{P1}[1] = 7 \rightarrow V_{P1}    V_{P2}$ , confirming concurrency. |
| TC13 | Global Snapshot 1                     | All 5 processes report STATE; P0 collects 5/5 and evaluates consistency                                                          | PASS — "All 5 process states collected for SNAPSHOT_1!"; verdict: CAUSALLY CONSISTENT.                                                                                                                                          |
| TC14 | Global Snapshot 2                     | Second snapshot triggered later in the run; all 5 processes report again under a distinct snapshot_id                            | PASS — "All 5 process states collected for SNAPSHOT_2!"; verdict: CAUSALLY CONSISTENT.                                                                                                                                          |
| TC15 | Graceful Shutdown                     | SIGINT/SIGTERM closes each process's listening socket cleanly                                                                    | PASS                                                                                                                                                                                                                            |

## 5. Additional Features Added (Beyond the Problem Statement)

- Colored, categorized audit logs (colorama) — [SEND] green, [RECV] cyan, [INTERNAL] yellow, [SNAPSHOT]/[MARKER] magenta — for readability during the live demo.
- Dynamic multi-node network configuration (config/hosts.cfg) — process→port and process→node-IP mapping resolved at startup, so the same code runs unmodified whether all 5 processes are on one machine or spread across multiple cluster nodes.
- Graceful shutdown handling (SIGINT/SIGTERM) on every process, cleanly closing listening sockets.
- Automated end-to-end demo mode (p0.py --auto) — a single command drives both orders and both snapshots without manual interaction, plus an interactive CLI menu for manual/ad-hoc testing.
- Causal consistency verification — P0 automatically checks, for every pair of collected process states, whether either process's recorded clock reflects information the other process hadn't yet recorded itself (for all  $i, j$ :  $V_i[j] \leq V_j[j]$ ), and reports the verdict alongside the snapshot table.
- In-transit channel-message recording for the Chandy-Lamport algorithm (record\_channel\_message), tracking messages that arrive on a channel while a snapshot is in progress.
- Two snapshots instead of one — captured SNAPSHOT\_1 mid-flight, while Order #101 had already been delivered by P3 but Order #102 was still mid-preparation at P2/P4, and SNAPSHOT\_2 after both delivery partners had successfully completed their deliveries, to demonstrate the algorithm capturing a consistent global state at two structurally different points in the workflow rather than just once.

## 6. Is the Captured Global State Consistent?

A recorded global snapshot  $\{V_{P0}, V_{P1}, V_{P2}, V_{P3}, V_{P4}\}$  is causally consistent if no recorded process state depends on an event that happened after another process's own recorded cut — formally, for every pair  $(i, j)$ :  $V_i[j] \leq V_j[j]$ . If this fails for some pair, process  $i$ 's snapshot reflects a message from  $j$  that, per  $j$ 's own recorded state, hadn't been sent yet — an impossible "future dependency."

## SNAPSHOT\_1

Triggered during concurrent processing of both orders (Order #101 already delivered by P3, Order #102 still mid-preparation at P2/P4 at the moment of the cut).

Recorded clocks:  $V_{P0}=[5,0,0,0,0]$ ,  $V_{P1}=[5,7,0,0,0]$ ,  $V_{P2}=[5,0,5,0,0]$ ,  $V_{P3}=[5,7,0,4,0]$ ,  $V_{P4}=[5,0,5,0,2]$ .

Verdict (program output): "Global Snapshot SNAPSHOT\_1 is CAUSALLY CONSISTENT! (No orphan or future dependencies)"

Manually verifying all 10 pairs against  $V_i[j] \leq V_j[j]$  confirms this holds in both directions for every pair — e.g.  $V_{P4}[2]=5$  equals  $V_{P2}[2]=5$  (P4 learned of P2 through the DELIVERY message, exactly matching P2's own recorded progress, not exceeding it), and every other cross-index comparison is a strict  $\leq$ . No process's recorded state depends on a not-yet-recorded event elsewhere.

## SNAPSHOT\_2

Triggered after both orders were dispatched and both deliveries had completed (post-workflow cut).

Recorded clocks:  $V_{P0}=[14,8,8,7,7]$ ,  $V_{P1}=[14,10,8,7,7]$ ,  $V_{P2}=[14,8,10,7,7]$ ,  $V_{P3}=[14,10,8,9,7]$ ,  $V_{P4}=[14,8,10,7,9]$ .

Verdict (program output): "Global Snapshot SNAPSHOT\_2 is CAUSALLY CONSISTENT! (No orphan or future dependencies)" — again confirmed by manual pairwise verification.

## On channel state

In this topology, every process has exactly one incoming edge ( $P0 \rightarrow P1$ ,  $P0 \rightarrow P2$ ,  $P1 \rightarrow P3$ ,  $P2 \rightarrow P4$ ). Chandy-Lamport always records the channel a process's first marker arrives on as empty, by definition of the algorithm — a non-empty in-transit channel recording is only possible for a process with two or more incoming edges receiving markers at different times, which doesn't occur here. So the empty recorded channel state in both of this system's snapshots is the mathematically expected, correct outcome of this topology, not a gap in the implementation.

## 7. Screenshots and Logs

Full raw terminal output from the master-node run is available in `outputs/logs/master_node_run.log` in the GitHub repository.

### 1. Shows P0 starting up and dispatching Order #101 and Order #102 to the two restaurants.

```
Applications: bits-food-delivery - Thu... Terminal - s2025mt03510@ip-10-3-25-219:~/workspace
Terminal - s2025mt03510@ip-10-3-25-219:~/workspace
File Edit View Terminal Tabs Help
s2025mt03510@ip-10-3-25-219:~/workspace/bits-food-delivery$ python3 src/p0.py --auto

=====
[P0] Central Order Processor listening on port 5005
Hosts: {'P0': '127.0.0.1', 'P1': '127.0.0.1', 'P2': '127.0.0.1', 'P3': '127.0.0.1', 'P4': '127.0.0.1'}
Initial Clock: [0, 0, 0, 0, 0]
=====

STARTING P0 AUTOMATED DISTRIBUTED WORKFLOW
=====

[STEP 1] Dispatching Order #101 to P1 (Pizza Palace)...
[INTERNAL] P0 | Created Customer Order #101 for Margherita Pizza | Clock: [1, 0, 0, 0, 0]
[SEND] P0 | Clock: [2, 0, 0, 0, 0]
[P0 Central Processor] DISPATCH ORDER #101 (Margherita Pizza) -> P1 | Clock: [2, 0, 0, 0, 0]

[STEP 2] Dispatching Order #102 to P2 (Burger Hub)...
[INTERNAL] P0 | Created Customer Order #102 for Double Cheeseburger | Clock: [3, 0, 0, 0, 0]
[SEND] P0 | Clock: [4, 0, 0, 0, 0]
[P0 Central Processor] DISPATCH ORDER #102 (Double Cheeseburger) -> P2 | Clock: [4, 0, 0, 0, 0]

[STEP 3] Triggering SNAPSHOT_1 during concurrent order processing...
```



2. Shows P0 collecting all 5 process states for SNAPSHOT\_1 and printing the causal consistency verdict.

```
=====
[SNAPSHOT INITIATION] P0 triggering SNAPSHOT_1
=====
[SNAPSHOT] P0 | SNAPSHOT_1 triggered | Clock: [4, 0, 0, 0, 0]
[SEND] P0 | Clock: [5, 0, 0, 0, 0]
[MARKER] P0 -> P1 | SNAPSHOT_1
[MARKER] P0 -> P2 | SNAPSHOT_1
[RECV] P0 | Clock: [6, 8, 0, 0, 0]
[STATE RECEIVED] P1 -> P0 | SNAPSHOT_1
[STATE RECEIVED] P1 -> P0 for SNAPSHOT_1 | Progress: 2/5 processes recorded (P0, P1)
[RECV] P0 | Clock: [7, 8, 6, 0, 0]
[STATE RECEIVED] P2 -> P0 | SNAPSHOT_1 [RECV] P0 | Clock: [8, 8, 6, 4, 0]

[STATE RECEIVED] P3 -> P0 | SNAPSHOT_1
[STATE RECEIVED] P3 -> P0 for SNAPSHOT_1 | Progress: 4/5 processes recorded (P0, P1, P2, P3)
[STATE RECEIVED] P2 -> P0 for SNAPSHOT_1 | Progress: 4/5 processes recorded (P0, P1, P2, P3)
[RECV] P0 | Clock: [9, 8, 6, 4, 2]
[STATE RECEIVED] P4 -> P0 | SNAPSHOT_1
[STATE RECEIVED] P4 -> P0 for SNAPSHOT_1 | Progress: 5/5 processes recorded (P0, P1, P2, P3, P4)
★ All 5 process states collected for SNAPSHOT_1!

--- Global Snapshot Report: SNAPSHOT_1 ---
Process    Recorded Vector Clock    Local State Summary
-----
P0          [5, 0, 0, 0, 0]           Active Orders: 2
P1          [5, 7, 0, 0, 0]           Active Orders: 0
P2          [5, 0, 5, 0, 0]           Active Orders: 0
P3          [5, 7, 0, 4, 0]           Status: PICKED_UP, Order: Orde
P4          [5, 0, 5, 0, 2]           Status: WAITING_FOR_ORDER, Ord

✓ Global Snapshot SNAPSHOT_1 is CAUSALLY CONSISTENT! (No orphan or future dependencies)
```

3. Shows both delivery confirmations, the SNAPSHOT\_2 report, and P0 shutting down after workflow completion.

```
[RECV] P0 | Clock: [10, 8, 6, 7, 2]
[INTERNAL] P0 | Delivery confirmation received from P3: Order #101 (Margherita Pizza) delivered by P3 (Fleet Runner A) | Clock: [11, 8, 6, 7, 2]
[DELIVERY COMPLETE] Order fulfilled via P3: Order #101 (Margherita Pizza) delivered by P3 (Fleet Runner A)
[RECV] P0 | Clock: [12, 8, 8, 7, 7]
[INTERNAL] P0 | Delivery confirmation received from P4: Order #102 (Double Cheeseburger) delivered by P4 (Fleet Runner B) | Clock: [13, 8, 8, 7, 7]
[DELIVERY COMPLETE] Order fulfilled via P4: Order #102 (Double Cheeseburger) delivered by P4 (Fleet Runner B)

[STEP 4] Triggering SNAPSHOT_2 post order workflow...

=====
[SNAPSHOT INITIATION] P0 triggering SNAPSHOT_2
=====
[SNAPSHOT] P0 | SNAPSHOT_2 triggered | Clock: [13, 8, 8, 7, 7]
[SEND] P0 | Clock: [14, 8, 8, 7, 7]
[MARKER] P0 -> P1 | SNAPSHOT_2
[MARKER] P0 -> P2 | SNAPSHOT_2
[RECV] P0 | Clock: [15, 11, 8, 7, 7]
[STATE RECEIVED] P1 -> P0 | SNAPSHOT_2
[STATE RECEIVED] P1 -> P0 for SNAPSHOT_2 | Progress: 2/5 processes recorded (P0, P1)
[RECV] P0 | Clock: [16, 11, 11, 7, 7]
[STATE RECEIVED] P2 -> P0 | SNAPSHOT_2
[STATE RECEIVED] P2 -> P0 for SNAPSHOT_2 | Progress: 3/5 processes recorded (P0, P1, P2)
[RECV] P0 | Clock: [17, 11, 11, 9, 7]
[STATE RECEIVED] P3 -> P0 | SNAPSHOT_2
[STATE RECEIVED] P3 -> P0 for SNAPSHOT_2 | Progress: 4/5 processes recorded (P0, P1, P2, P3)
[RECV] P0 | Clock: [18, 11, 11, 9, 9]
[STATE RECEIVED] P4 -> P0 | SNAPSHOT_2
[STATE RECEIVED] P4 -> P0 for SNAPSHOT_2 | Progress: 5/5 processes recorded (P0, P1, P2, P3, P4)
★ All 5 process states collected for SNAPSHOT_2!

--- Global Snapshot Report: SNAPSHOT_2 ---
Process    Recorded Vector Clock    Local State Summary
-----
P0          [14, 8, 8, 7, 7]           Active Orders: 2
P1          [14, 10, 8, 7, 7]           Active Orders: 0
P2          [14, 8, 10, 7, 7]           Active Orders: 0
P3          [14, 10, 8, 9, 7]           Status: DELIVERED, Order: Orde
P4          [14, 8, 10, 7, 9]           Status: DELIVERED, Order: Orde

✓ Global Snapshot SNAPSHOT_2 is CAUSALLY CONSISTENT! (No orphan or future dependencies)

=====
AUTOMATED WORKFLOW COMPLETED - P0 READY
=====
```

4. Shows P1's full event log: order confirmation, food prep, and delivery handoff to P3.

```
Applications ▾ bits-food-delivery - Thu... Xfce Terminal
Terminal - s2025

File Edit View Terminal Tabs Help

s2025mt03510@ip-10-3-25-219:~/workspace/bits-food-delivery$ python3 src/p1.py
[P1 Pizza Palace] listening on port 5001 (Host: 127.0.0.1) ...
[RECV] P1 | Clock: [2, 1, 0, 0, 0]
[P1 Pizza Palace] RECEIVE ORDER 101 from P0 | Clock: [2, 1, 0, 0, 0]
[INTERNAL] P1 | Order confirmed | Clock: [2, 2, 0, 0, 0]
[INTERNAL] P1 | Preparing food | Clock: [2, 3, 0, 0, 0]
[INTERNAL] P1 | Food ready for pickup | Clock: [2, 4, 0, 0, 0]
[SEND] P1 | Clock: [2, 5, 0, 0, 0]
[RECV] P1 | Clock: [5, 6, 0, 0, 0]
[MARKER RECV] P1 <- P0 | SNAPSHOT_1 | Clock: [5, 6, 0, 0, 0]
[SEND] P1 | Clock: [5, 7, 0, 0, 0]
[MARKER] P1 -> P3
[SEND] P1 | Clock: [5, 8, 0, 0, 0]
[STATE] P1 -> P0 | SNAPSHOT_1
[RECV] P1 | Clock: [14, 9, 8, 7, 7]
[MARKER RECV] P1 <- P0 | SNAPSHOT_2 | Clock: [14, 9, 8, 7, 7]
[SEND] P1 | Clock: [14, 10, 8, 7, 7]
[MARKER] P1 -> P3
[SEND] P1 | Clock: [14, 11, 8, 7, 7]
[STATE] P1 -> P0 | SNAPSHOT_2
```

5. Shows P2's full event log: order confirmation, food prep, and delivery handoff to P4.

```
Applications ▾ bits-food-delivery - Thu... Xfce Terminal
Terminal - s2025

File Edit View Terminal Tabs Help

s2025mt03510@ip-10-3-25-219:~/workspace/bits-food-delivery$ python3 src/p2.py
[P2 Burger Hub] listening on port 5002 (Host: 127.0.0.1) ...
[RECV] P2 | Clock: [4, 0, 1, 0, 0]
[P2 Burger Hub] RECEIVE ORDER 102 from P0 | Clock: [4, 0, 1, 0, 0]
[INTERNAL] P2 | Order confirmed | Clock: [4, 0, 2, 0, 0]
[INTERNAL] P2 | Preparing food | Clock: [4, 0, 3, 0, 0]
[RECV] P2 | Clock: [5, 0, 4, 0, 0]
[MARKER RECV] P2 <- P0 | SNAPSHOT_1 | Clock: [5, 0, 4, 0, 0]
[SEND] P2 | Clock: [5, 0, 5, 0, 0]
[MARKER] P2 -> P4
[SEND] P2 | Clock: [5, 0, 6, 0, 0]
[STATE] P2 -> P0 | SNAPSHOT_1
[INTERNAL] P2 | Food ready for pickup | Clock: [5, 0, 7, 0, 0]
[SEND] P2 | Clock: [5, 0, 8, 0, 0]
[RECV] P2 | Clock: [14, 8, 9, 7, 7]
[MARKER RECV] P2 <- P0 | SNAPSHOT_2 | Clock: [14, 8, 9, 7, 7]
[SEND] P2 | Clock: [14, 8, 10, 7, 7]
[MARKER] P2 -> P4
[SEND] P2 | Clock: [14, 8, 11, 7, 7]
[STATE] P2 -> P0 | SNAPSHOT_2
```





6. Shows P3's full event log: pickup, in-transit, delivery, and completion report to P0.

```
Applications ▾ bits-food-delivery - Thu... Xfce Terminal
Terminal - s2025mt03510@ip-10-3-25-219:~/workspace/

File Edit View Terminal Tabs Help

s2025mt03510@ip-10-3-25-219:~/workspace/bits-food-delivery$ python3 src/p3.py
=== P3 : Delivery Partner 1 (Fleet Runner A) starting up ===
P3 (Fleet Runner A) listening on port 5003 (Host: 127.0.0.1)...
[RECV] P3 | Clock: [2, 5, 0, 1, 0]
[RECV] P3 ← P1 | Order #101 (Margherita Pizza) handed off for delivery | Clock: [2, 5, 0, 1, 0]
[INTERNAL] P3 | Order #101 (Margherita Pizza) picked up from P1 | Clock: [2, 5, 0, 2, 0]
[RECV] P3 | Clock: [5, 7, 0, 3, 0]
[RECV] P3 ← P1 | MARKER for SNAPSHOT_1 | Clock: [5, 7, 0, 3, 0]
[SNAPSHOT] P3 | SNAPSHOT_1 local state recorded (order_status=PICKED_UP) | Clock: [5, 7, 0, 3, 0]
[SEND] P3 | Clock: [5, 7, 0, 4, 0]
[SEND] P3 → P0 | STATE for SNAPSHOT_1 | Clock: [5, 7, 0, 4, 0]
[INTERNAL] P3 | Order #101 (Margherita Pizza) in transit (Fleet Runner A) | Clock: [5, 7, 0, 5, 0]
[INTERNAL] P3 | Order #101 (Margherita Pizza) delivered to customer | Clock: [5, 7, 0, 6, 0]
[SEND] P3 | Clock: [5, 7, 0, 7, 0]
[SEND] P3 → P0 | Order #101 (Margherita Pizza) delivered by P3 (Fleet Runner A) | Clock: [5, 7, 0, 7, 0]
[RECV] P3 | Clock: [14, 10, 8, 8, 7]
[RECV] P3 ← P1 | MARKER for SNAPSHOT_2 | Clock: [14, 10, 8, 8, 7]
[SNAPSHOT] P3 | SNAPSHOT_2 local state recorded (order_status=DELIVERED) | Clock: [14, 10, 8, 8, 7]
[SEND] P3 | Clock: [14, 10, 8, 9, 7]
[SEND] P3 → P0 | STATE for SNAPSHOT_2 | Clock: [14, 10, 8, 9, 7]
```

7. Shows P4's full event log: pickup, in-transit, delivery, and completion report to P0.

```
Applications ▾ bits-food-delivery - Thu... Xfce Terminal
Terminal - s2025mt03510@ip-10-3-25-219:~/workspace/bits-f

Applications
File Edit View Terminal Tabs Help

s2025mt03510@ip-10-3-25-219:~/workspace/bits-food-delivery$ python3 src/p4.py
=== P4 : Delivery Partner 2 (Fleet Runner B) starting up ===
P4 (Fleet Runner B) listening on port 5004 (Host: 127.0.0.1)...
[RECV] P4 | Clock: [5, 0, 5, 0, 1]
[RECV] P4 ← P2 | MARKER for SNAPSHOT_1 | Clock: [5, 0, 5, 0, 1]
[SNAPSHOT] P4 | SNAPSHOT_1 local state recorded (order_status=WAITING_FOR_ORDER) | Clock: [5, 0, 5, 0, 1]
[SEND] P4 | Clock: [5, 0, 5, 0, 2]
[SEND] P4 → P0 | STATE for SNAPSHOT_1 | Clock: [5, 0, 5, 0, 2]
[RECV] P4 | Clock: [5, 0, 8, 0, 3]
[RECV] P4 ← P2 | Order #102 (Double Cheeseburger) handed off for delivery | Clock: [5, 0, 8, 0, 3]
[INTERNAL] P4 | Order #102 (Double Cheeseburger) picked up from P2 | Clock: [5, 0, 8, 0, 4]
[INTERNAL] P4 | Order #102 (Double Cheeseburger) in transit (Fleet Runner B) | Clock: [5, 0, 8, 0, 5]
[INTERNAL] P4 | Order #102 (Double Cheeseburger) delivered to customer | Clock: [5, 0, 8, 0, 6]
[SEND] P4 | Clock: [5, 0, 8, 0, 7]
[SEND] P4 → P0 | Order #102 (Double Cheeseburger) delivered by P4 (Fleet Runner B) | Clock: [5, 0, 8, 0, 7]
[RECV] P4 | Clock: [14, 8, 10, 7, 8]
[RECV] P4 ← P2 | MARKER for SNAPSHOT_2 | Clock: [14, 8, 10, 7, 8]
[SNAPSHOT] P4 | SNAPSHOT_2 local state recorded (order_status=DELIVERED) | Clock: [14, 8, 10, 7, 8]
[SEND] P4 | Clock: [14, 8, 10, 7, 9]
[SEND] P4 → P0 | STATE for SNAPSHOT_2 | Clock: [14, 8, 10, 7, 9]
```





## 8. Conclusion

This assignment made distributed computing concepts click in a way theory alone didn't. Working with `vector_clock.py` showed the real difference between scalar and vector clocks, since only a vector can tell you two events are truly concurrent, which TC12 proved directly. Building the Chandy-Lamport snapshot in `snapshot.py` made clear how a snapshot can be causally consistent without being a real single instant in time, as shown by `SNAPSHOT_1` catching one order already delivered while the other was still being prepared. As a team, splitting the system into five processes forced us to agree on clean interfaces and share one `vector_clock.py` and `snapshot.py` instead of each writing our own. When the Prayogshala inter-node ports weren't open in time, we adjusted and ran single-node instead of letting infrastructure block the point of the exercise.

## 9. References

- K. Mani Chandy and Leslie Lamport, "Distributed Snapshots: Determining Global States of Distributed Systems," ACM TOCS, 1985.
- Colin Fidge, "Timestamps in Message-Ordering Systems," Proc. 11th Australian Computer Science Conference, 1988.